

Batched Graph Community Detection on GPUs

Han-Yi Chou*
NVIDIA
hanyic@nvidia.com

Sayan Ghosh
Pacific Northwest National Laboratory
sg0@pnnl.gov

Abstract

Graph clustering or community detection is a novel method to identify closely related structures within a graph. Applications of graph clustering can be found in cybersecurity, computational biology, proteomics, power grids, et al. As such, developing high performance implementations of graph clustering can enhance analytics pipelines. Currently, GPUs are an integral part of HPC systems, especially dense-GPU systems are ubiquitous. Hence, there is a significant effort to port applications on GPUs to extract maximal performance from modern single-node HPC platforms. In general, designing graph applications efficiently on GPUs is challenging, due to relatively low arithmetic computational intensity and arbitrarily high irregular memory accesses.

The graph clustering problem is inherently serial, and the current modularity-maximization based parallel graph clustering implementations usually incorporate additional heuristics such as distance- k coloring to improve the overall quality and convergence. In contrast, we introduce a batched method for clustering on GPUs that allow users to achieve the desirable amount of quality without compromising the performance and involving sophisticated algorithms/heuristics. Due to a partition-based design, we can process graphs larger than the combined GPU memory of a node. Despite of increasing the frequency of synchronization through batches (batches are processed in a bulk-synchronous manner among GPUs), the effect on the performance is mild for large graphs.

Our implementation is based on the popular Louvain method for graph clustering, and we compare our performance and quality with existing parallel Louvain implementations on CPU and GPU systems. Overall, we observed speedups of about $2\text{--}14\times/1.3\text{--}7.5\times$ on up to 16/8 NVIDIA V100/A100 GPUs and a maximum speedup of $50\times/30\times$ as compared to state-of-the-art shared-memory and GPU-based Louvain implementations. We also demonstrate competitive if not better quality as compared to the existing parallel implementations of Louvain using tens to hundreds of batches.

*This work was performed when the author was a graduate student in University of Illinois, Urbana, IL, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

PACT '22, October 8–12, 2022, Chicago, IL, USA

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-9868-8/22/10...\$15.00

<https://doi.org/10.1145/3559009.3569655>

CCS Concepts

• **Computing methodologies** → **Shared memory algorithms**;
• **Theory of computation** → **Graph algorithms analysis**; •
General and reference → **Performance**;

Keywords

Graph Community Detection, Louvain method, GPU, CUDA

1 Introduction

Graph clustering or community detection is the problem of partitioning the vertices of a connected graph or network into subsets called communities or clusters such that vertices within a community are tightly connected with each other, whereas vertices across the communities are sparsely connected. Modularity [38] is a metric to measure the quality of clustering. Algorithms and heuristics based on modularity optimization are proved to be NP-hard [7]. Relevant discussions on the various methods and applications of community detection can be found in [12, 18, 37, 41]. In this paper, we focus on a relatively popular modularity optimization based heuristic which is known to outperform other community detection methods, namely Louvain method [4]. We discuss a scalable multi-GPU implementation of the Louvain method with options to control the quality on contemporary dense-GPU systems.

HPC systems with multiple GPUs per node are common, and this trend seems to continue in the near future. For instance, seven out of ten top contemporary supercomputers in the November 2021 Top500 list have high-end NVIDIA™ GPUs [14]. Upcoming U.S. Exascale supercomputers are also equipped with state-of-the-art GPUs [29]. Graph algorithms/heuristics are usually part of existing data analytics and scientific computing pipelines (such as NVIDIA RAPIDS [26], Tridata [50], Apache Spark [46] etc.) and have the potential to be embedded into a larger “converged” application context. Thus, it is crucial to investigate efficient GPU implementations for common graph algorithms such as clustering. However, developing scalable graph applications is often challenging due to irregular data-access intensive nature of the problems [34, 35]. The non-deterministic aspect of parallel graph heuristics can also lead to performance variability and inconsistent quality issues. Louvain method is inherently sequential, and parallel updates can lead to negative gains and local maxima scenarios [33]. Graph coloring has been used traditionally to enhance the quality of graph applications, and in case of clustering it enables sequentialization of adjacent vertex updates (adjacent vertices have different colors, such that vertices of the same color can be processed together) that can mitigate some of the issues caused by parallelism. However, coloring requires reordering of the vertices by colors and potentially extra synchronization (once per color set) and memory requirements. A relatively simpler alternative to enforce sequentialization is to consider batches of the adjacent vertices in a graph and process them

consecutively. This is the fundamental idea behind our design—we divide a graph into partitions (based on the maximum partition size that can fit into a GPU), and then divide the computations on a partition into successive batches.

The advent of high-speed interconnects such as NVLink [16] and PCIe has led to dense single-node multi-GPU systems such as Cray CS-Storm, HPE Apollo/SGI and NVIDIA DGX. Affordable pricing of such dense-GPU nodes by the cloud vendors will further improve their accessibility. This is an important development because it provides an option to continue using the shared-memory programming models for running relatively larger workloads, which otherwise would have required the more complicated distributed-memory models for scaling to multiple nodes. Distributed-memory graph problems in particular suffer from communication overheads at scale, despite attempts to optimize the graph partitioning (which is NP-hard [8]). In this paper, we discuss the design, implementation and performance of our intuitive parallel batched Louvain graph clustering on multiple GPUs. While recent GPU implementations of graph clustering have exclusively focused on improving the performance relative to CPU implementations, our goals are to investigate options that can enhance the quality without compromising scalability. Particularly, our contributions are:

- Multi-GPU implementation of the popular *Louvain method* for graph community detection [4], using CUDA and OpenMP, leveraging CUDA Unified Virtual Addressing (UVA) for direct access to the peer device memories.
- Designed a methodology to update the vertex communities in user defined batches, as an alternative to popular coloring and labeling based heuristics: trade-off performance with quality (batches are processed consecutively within a GPU and in parallel across GPUs, with host-device synchronization at the end of every batch).
- Adaptive partitioning allows large graphs (which cannot fit into the memory of a single GPU) to be processed on a single GPU.
- Extensive performance evaluations and Louvain convergence analysis using a variety of real-world graphs on the latest NVIDIA™ GPU platforms (i.e., DGX-2™ and DGX-A100™).
- Performance and quality comparisons with state-of-the-art CPU (GRAPPOLO [33]) and GPU (NVIDIA™ RAPIDS™ cuGRAPH [26]) Louvain implementations.

The rest of the paper is organized as follows. We provide a brief background and motivation for our work in §2. §3 outlines our design and methodology. §4 analyzes the scalability/convergence characteristics of our batched graph clustering implementation on the GPU testbeds, and compares the performance/quality with existing CPU/GPU implementations. We discuss related approaches and associated results in §5. Finally, §6 concludes the paper.

2 Background and Motivation

Consider an undirected graph $G = (V, E, w)$ which is a collection of a vertex set, V , an edge set, E with weights w associated with the edges. Modularity [38] is a measure of the quality of partitioning of a graph into k communities, denoted by $C = \{C_1, C_2, C_3, \dots, C_k\}$. Modularity (denoted by Q) maximization is an NP-hard problem [7]

and the formulation is shown in Equation 1.

$$Q = \sum_{c \in C} \left(\frac{e_{ij}}{2m} - \frac{a_c^2}{4m^2} \right) \quad (1)$$

where $e_{ij} = \sum_{i,j \in c} w_{ij}$, $m = \sum_{e \in E} \omega(e)$, and $a_c = \sum_{i \in c} k_i$ (sum of degree of all vertices in cluster c). Several algorithms and heuristics have been proposed to approximate the optimal partition [18].

Louvain Method Among existing algorithms, the Louvain method [4] is a relatively simple yet efficient technique for estimating the optimal clustering of a graph. Louvain is a greedy iterative optimization method consisting of multiple phases, and each phase runs for multiple iterations/loops until convergence. Initially, each vertex is assigned to a separate community (singleton partition). Broadly, the underlying computations of Louvain fall into two categories—the *local (vertex) moving phase* and the *graph compaction phase*. In the local moving phase of an iteration, each vertex is assigned to a neighboring community to achieve the greatest modularity gain. The gain in modularity (denoted by ΔQ) when moving a vertex i to a neighboring cluster C_j (vertices i and j are adjacent) in a given community/cluster set C is represented by Equation 2.

$$\Delta Q(i \rightarrow C_j) = \frac{e_{i \rightarrow C_j} - e_{i \rightarrow C_i \setminus \{i\}}}{m} - \frac{k_i(a_{C_i \setminus \{i\}} - a_{C_j})}{2m^2} \quad (2)$$

where $e_{i \rightarrow C_j}$ is the set of edges e_{ij} with vertex j in cluster C_j and a_{C_i} is the sum of degree of all vertices in cluster C_i . Note that the above expressions remain valid for multigraphs and graphs with self loops. After the local moving phase is complete (i.e., the ΔQ between successive iterations falls below a threshold), vertices within the same community/cluster are merged into a single vertex and weights of the corresponding incident edges are combined (i.e., *graph compaction*), creating the graph for the next phase (as shown in Fig. 1 (a)). When the modularity gain (i.e., ΔQ) across successive phases falls below a threshold, convergence is achieved.

Issues with parallel implementations of Louvain method

In the sequential implementation of the Louvain algorithm [4], each vertex is visited consecutively and the target community is determined by maximizing the ΔQ in Equation 2. In such a manner, the characteristic of greedy algorithm manifests and the change of modularity is guaranteed to be non-negative in every vertex moving phase [7]. However, in a parallel implementation of the Louvain method, the gain of modularity could be negative as multiple vertices update their communities concurrently based on the same partitioning configuration and outdated information (for performance reasons, synchronization is infrequent). This can negatively affect the convergence [33]. Since the Louvain method is inherently serial, parallel implementations usually impose heuristics to trade-off some performance for quality. Graph coloring is a way to mitigate some of the issues by enforcing serialization, but coloring is hard and expensive. Other strategies include labeling schemes to avoid local maxima.

To address some of the issues arising from parallelism, one of the popular choice is to adopt the k -vertex coloring heuristics. The k -vertex coloring method label vertices such that any two vertices possess different colors when their separation is not larger than k . The vertices labeled with the same colors can be processed in parallel. However, coloring heuristics usually involve overheads

when applied to graph clustering, such as requisite synchronization after updating a color batch. Also, there is a trade-off between minimizing the number of colors (optimal coloring is NP-complete [20] and there are no guaranteed upper bounds on #colors) and performance in graph coloring algorithms [21]. Depending on the choice of the coloring algorithm/heuristic (a number of them are inherently sequential) and the complexity of the task that coloring aims to optimize, coloring can be computationally expensive. For instance, greedy coloring algorithms might additionally require re-ordering the vertices of the graph by their degrees [48]. Moreover, variants of parallel independent set based coloring heuristics have demonstrated limited parallel efficiency [1].

Our solution In this paper, we present a simple yet efficient heuristic called *batched vertex update*. Compared to the coloring method, there is no preference order nor labeling of vertices. The vertices are divided into several user-defined batches and each batch is updated in a bulk-synchronous fashion. To the best of our knowledge, we are not aware of existing studies using the proposed *batched vertex* scheme in the context of parallel Louvain method for graph clustering.

However, similar concepts exist. In [11], the authors propose a hierarchical algorithm and divides the nodes/vertices per-level into fixed-size batches such that nodes within a batch is processed in parallel. In [45], authors introduce the notion of *chunking* for moving appropriately-sized chunks of vertices to high-bandwidth memory (HBM) to accelerate the label propagation algorithm. In [6] the authors study graph coloring on batches of input vertices for certain types of graphs. Classical combinatorial problems such as bin packing have also been studied previously for batches of items [24].

3 Louvain Method over Batches

We discuss the design of our *batched* Louvain method in this section. We use the Compress Sparse Row (CSR) data structure to represent graphs on device. The vertices of the graph have unique consecutive IDs. Hence, subgraph on a device have local consecutive vertex IDs to utilize coalesced accesses on device global memory. We provide vertex or edge based partitioning of the graph among devices, but for this paper we ultimately used the edge-based partitioning scheme as it offers better load balance. Threads pinned to different NUMA sockets read portions of the input graph to enable first-touch access. Threads forked in an OpenMP parallel region controls a device.

This section consists of four parts: mapping graph to GPU threads (§3.1), batching and accessing data on devices (§3.2), the mechanism of updating communities (§3.3) and comparing batching with common labeling heuristics in the context of graph clustering (§3.4).

3.1 Mapping graph to CUDA thread blocks

Traditionally, synchronization only consisted of a (thread) block level barrier in CUDA. Recently, *Cooperative Groups* [25] provides a flexible model for synchronization and communication within and across dynamic groups of threads. We make use of cooperative groups (of sizes 16/32) in mapping the graph to multiple tiles of 32 threads (*warp*), allowing us to invoke warp-collective operations

like *shuffle* on the tiles to simplify reductions¹ (bypassing scarce shared memory for storing partial results). Each thread block is divided into several *cooperative thread groups*, depicted by different colors in the Fig. 2, and each group is assigned to a vertex. The incident edges to the vertex are mapped to the threads according to the edge index modulo to the size of the particular thread group.

3.2 Partitions, batches and multiple devices

Each subgraph on the device can be divided into multiple batches as shown in Fig. 1 (b). Since GPU memory is limited and data transfers are expensive, we devise a mechanism to reduce the amount of CUDA memory copies in the vertex moving phase. We define a partition size, P_s , as the maximum number of edges that could be stored on a device. Before entering the Louvain iteration, the available device memory is inquired (via `cudaMemGetInfo` CUDA runtime API function) and based on it the partition size P_s is estimated. If the batch size is smaller than P_s , we copy P_s edges (and related weights) to the device. If the batch size is larger than P_s , the batch is subdivided into portions $< P_s$ and Louvain updates are performed on these portions one at a time, as shown in Fig. 1 (d). We also use 2–4 CUDA streams per device to concurrently copy the graph partitions (associated edge lists and weights).

Batches are processed in parallel across the devices. The local vertex moving phase determines the optimal clusters for each vertices in a batch. Before updating the vertex-community association, we perform synchronization for the host threads and the device kernels. We frequently require accessing the community information (i.e., weights, community associated with a vertex) owned by a different device; since a vertex can migrate to a neighboring community which is physically located in the address space of another device as shown in Fig. 1 (c). Hence, we use *binary search* to quickly locate the desired device for a given global vertex/community index. Our methodology is captured in Algorithm 1.

We take advantage of the Unified Virtual Addressing (UVA) technology provided by contemporary NVIDIA platforms to access and update the data structures associated with the address space of a remote device (enabling this functionality through appropriate peer device memory access functions, such as `cudaDeviceEnablePeerAccess`). UVA makes it possible for CUDA to use a *single virtual address space* for host and device memory allocations. As a consequence of unified addressing, there is no distinction between host and device pointers, and the underlying location of memory corresponding to a pointer can be assessed by the runtime as long as page-locked/pinned (a.k.a managed) memory is used. Modern NVIDIA GPUs also have support for automatic migration of memory pages upon a page fault (via *unified memory*, eliminating the need for explicit data movement), which the UVA technology enables (*UVA* and *unified memory* are otherwise distinct concepts).

3.3 Updating communities

The most consequential part of the Louvain method is to determine the optimal neighboring community/cluster for a vertex. To evaluate Equation 2, a mapping between neighboring vertices and the communities are established when traversing the incident edges to compute $e_{i \rightarrow c_j}$ for every possible c_j . Existing parallel implementations of the Louvain method [22, 33] have used associative

¹For key-value pair reductions, we use Thrust library, i.e., `thrust::reduce_by_key`.

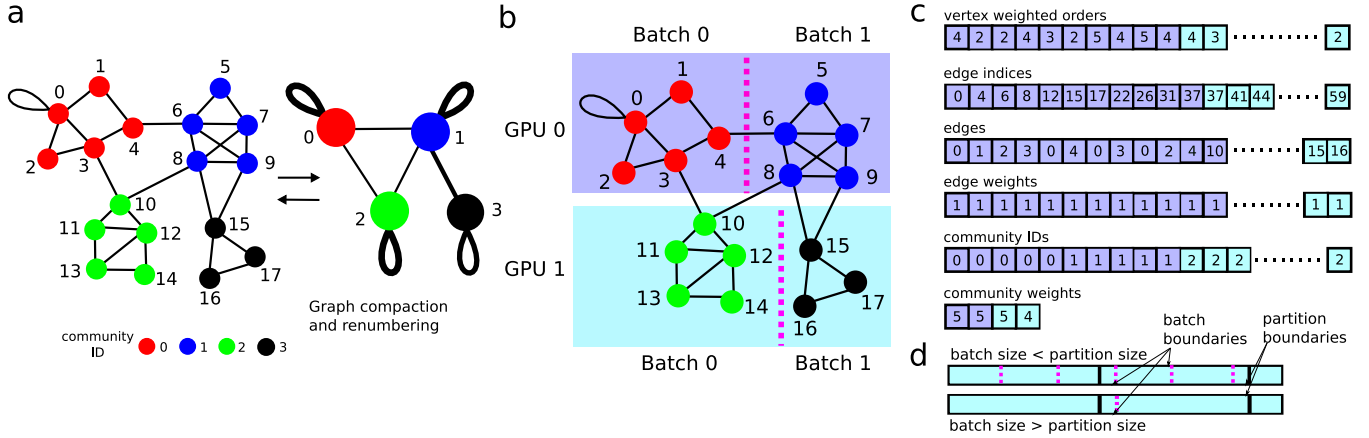


Figure 1: (a) An illustration of the Louvain algorithm on a sample graph input. In a Louvain phase, after each vertex is assigned its optimal cluster (indicated by different colors), vertices in the same community merge into a meta-vertex with new incident edges carrying the total edge weights (represented by thicker lines). Repeating the Louvain phase on the new compacted graph (b) The graph is divided on multiple GPUs (colored in the background by blue and cyan) and each sub-graph on the device is further divided into several batches (purple dash lines). (c) The memory layouts for the graph in CSR data representation. Each device possesses only fragments of the arrays to represent the associated edges, weights and the community indices of its subgraph. For the community weights, the array is evenly divided among devices. (d) Due to limited GPU memory size, each device is only able to process part of the edges at a time. We determined the maximum partition size of edges which can be accommodated on the GPU at a time. When the partition size (represented by the black line) is larger than the batch size (indicated by the dash purple line), the whole partition containing the current batch is copied to the device. Thus, when the computation proceeds to the next batch, there is no need to initiate a copy. On the other hand, a batch is processed partition by partition if the batch size exceeds the partition size.

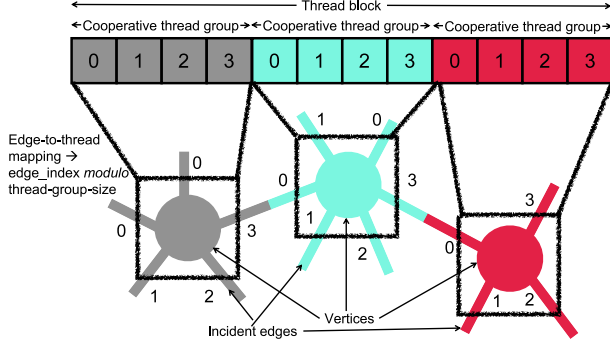


Figure 2: An example of mapping between the thread blocks and the graph vertices/edges with a 1D thread block.

C++ containers (e.g., `std::map` and `std::unordered_map`) with $O(\log n)$ lookup costs to optimize the frequent accesses.

We chose to store the incident edges in the order of their associated community indices for each vertex and maintain a CSR data structure to map a vertex to its corresponding memory address. The sorting is performed when the communities of vertices are reassigned after the local vertex moving phase for a batch. Another aspect of the batched design is that the sorting is only performed on the batch to be processed, instead of the entire graph. We used the Thrust CUDA library [2] to perform the sorting. A reduction by key on the edge weights using the Thrust library is performed on the ordered edge weights to obtain the $e_i \rightarrow C_j$ where i and j are adjacent vertices. A CSR data structure is constructed to hold the results

from the segmented reduction for each vertex. The maximum ΔQ is then straightforward to calculate for each vertex.

After determining the optimal moves for each vertex within a batch, we update their associated community index and weights via UVA. Since each device only possesses one segment of the community indices and weights, we use binary search to locate actual device owning the requisite data, as shown in Fig. 1 (c).

On completing the local moving phase on GPUs, we build the graph for the next phase (i.e., graph compaction), by aggregating and renumbering the edges. The compaction happens on the CPU (uses OpenMP) after the GPU sorts its vertices according to their associated community indices and copies the data back to the host. This sorting step on a GPU only accounts for a relatively small proportion in the total compaction time as the complexity only scales with the number of vertices. In contrast, the sorting step in the previous local moving phase scales with the number of edges. At this point, the device pointers are also reset for the next phase.

3.4 Contrasts with labeling heuristics

Louvain is inherently serial, and parallel updates to neighboring communities can lead to situations discussed in [33], which affects quality and performance. Heuristics mentioned in [33] are included in state-of-the-art parallel CPU implementation of Louvain clustering, namely Grappolo. Parallel clustering implementations cannot completely guarantee negative gain mitigation, even with graph coloring. Hence, most if not all existing approaches use relatively cheaper labeling schemes such as *minimum label heuristic* in their baseline versions to avoid the local maxima. According to the minimum label heuristic, if multiple neighboring communities can yield

Algorithm 1 Batched Louvain Method on multiple GPUs.

Input: $G(V, E, \omega)$ (in CSR format),

Input: Threshold, τ (default: 10^{-6}), #batches, #GPUs

Output: Community assignment C

```

1: while not done do {Phase}
2:    $C \leftarrow \{\{v\} | \forall v \in V\}$ 
3:    $Q \leftarrow Q(C)$  {Compute modularity}
4:    $Q_0 \leftarrow Q$ 
5:   #partitions  $\leftarrow \text{get\_num\_partitions}(G)$ 
6:   while not converged do {Iteration}
7:     begin{multi-device computation}
8:        $\text{omp\_set\_num\_threads}(\#\text{GPUs})$ 
9:       #pragma omp parallel {OpenMP region starts}
10:       $\text{SetDevice}(\text{omp\_get\_thread\_num}(\%) \cdot \#\text{GPUs})$ 
11:      for all  $p$  in #partitions do:
12:        Calculate graph partition ranges,  $\text{range}(p)$ 
13:        Copy graph partition to the current device
14:        for all  $b$  in #batches do:
15:          for all  $v$  in  $\text{range}(p, b)$  do:
16:             $c \leftarrow \arg\max_{c \in C} \Delta Q$ 
17:             $C[v] \leftarrow c$  {Target community}
18:          host/device synchronization
19:        {OpenMP region ends}
20:      end{multi-device computation}
21:       $Q \leftarrow Q(C)$ 
22:       $\Delta Q \leftarrow Q - Q_{\text{old}}$ 
23:      if  $\Delta Q \leq \tau$  then
24:         $C \leftarrow C_{\text{old}}$  {Final communities}
25:        converged {break from loop}
26:      else
27:         $(Q_{\text{old}}, C_{\text{old}}) \leftarrow (Q, C)$ 
28:      if  $Q - Q_0 \leq \tau$  then
29:        done
30:      else
31:        Copy vertices, weights and communities from devices
32:         $G \leftarrow$  Graph compaction and recalculate partitions

```

the maximum modularity gain, then a vertex chooses the community with the minimum label. This scheme can prevent vertices swapping their respective communities without resulting in any net modularity gain, thus delaying convergence.

NVIDIA RAPIDS CuGRAPH [39] and Gunrock [40] are contemporary GPU frameworks for graph analytics, that employs the minimum label heuristic. CuGRAPH incorporates the general minimum label heuristic² (from Grappolo [33]) to minimize the negative effects of local maxima, whereas Gunrock [40] uses the singlet minimum label heuristic (preventing swap scenarios for singlet community associations). Galois [30] is another framework for developing irregular applications, and currently includes CPU implementation of Louvain clustering. It also incorporates the minimum label heuristic, and additionally uses a parallel loop construct to enforce dependencies between multiple threads (essentially locking a vertex neighborhood while determining its target community).

²<https://github.com/rapidsai/cugraph/tree/v21.10.00/cpp/src/community>

We do not implement any other heuristic apart from *batching*. Batching does not prevent the swap scenario that minimum label heuristic solves, but relatively frequent synchronization (at the end of every batch) captures global modularity gains more accurately. Batching also has minimal overheads compared to graph coloring, which has to group the adjacent vertices into distinct color sets (synchronized per color set). But, as we observe in §4.2.2, apart from improving quality, batches can enhance the overall performance by reducing the #iterations to convergence (like coloring).

4 Results and Analysis

This section covers the following topics: our experimental CPU/GPU platforms (§4.1), multi-GPU baseline (single batch) performances of large graphs (§4.2.1), analysis of the impact of batches on convergence/performance (§4.2.2), profiling analysis of our batched implementation (§4.2.3), and single-GPU performance/quality comparisons with existing CPU/GPU Louvain implementations (GRAPPOLO/cuGRAPH) for some difficult datasets (§4.3).

4.1 Experimental platforms, software and data

4.1.1 Platforms: We use three platforms in our evaluations: NVIDIA™ DGX systems with 8×/16× GPUs (DGX-A100 and DGX-2) and IBM AC922 Power9 with 4× NVIDIA™ V100 GPUs. NVIDIA™ DGX-2 V100 platform consists of a single node with 16 “Volta” V100 GPUs (with 80 symmetric multiprocessors a.k.a SMs) with 32GB HBM2 memory/GPU and two-way 24-core Intel Xeon P-8168 CPUs (“Sky-Lake” or SKL) at 2.7GHz, 33MB L3 cache, 6 memory channels, and 1.5TB DDR4 memory. DGX A100 is the third generation server node from NVIDIA™, and consists of 8 “Ampere” A100 GPUs (with 108 SMs) with 40GB HBM2 memory/GPU and two-way 128-core AMD EPYC 7742 CPUs at 2.25GHz, 256MB L3 cache, 8 memory channels, and 1TB DDR4 memory. The Power9 platform is a dual-socket system consisting of two 24-core IBM Power9 CPUs at 4GHz and 10MB L3 cache per core-pair, 4 NVIDIA “Volta” V100 GPUs with 16GB of HBM2 memory and 1TB DDR4 memory. Volta/Ampere SMs have a unified shared memory and L1 data cache (128KB/192KB) which is configurable at runtime, and we set an option to prefer a larger L1 cache (via `cudaFuncCachePreferL1`). Comparative CPU evaluations (for OpenMP enabled CPU Louvain clustering code, GRAPPOLO [33]) were performed on an 8-way Intel Platinum 8276M (Cascade Lake, CXL) CPU at 2.20GHz with 38.5MB L3 cache and 6TB main memory, using LLVM 12.0 compiler.

4.1.2 Peer-to-peer Copy Performance: Since we use Unified Virtual Addressing (UVA) for directly accessing the memory of multiple devices (see §3.3), the interconnect bandwidth is crucial for performance. Typically high-end NVIDIA GPUs either come in the PCIe or proprietary NVLink/NVSwitch mesh based form factors [32]. Recently, NVIDIA introduced the SXM module that allows GPUs to directly communicate with other GPUs/CPU through NVLink.

Table 1: Bandwidth (GB/s) differences on multiple GPUs. Benchmark performs vector copy through UVA.

Platforms/#GPUs	1	2	4	8
Volta V100-SXM2 (16GB)	663.8	112.6	55.1	-
Volta V100-SXM3 (32GB)	696.7	225.6	224.1	222.1
Ampere A100-SXM4 (40GB)	946.3	310.9	305.2	303.9

The bandwidth results for a simple vector copy benchmark (i.e., $a[i] = b[i]$) where the vectors are located on distinct peer devices (writing to neighboring device memory) demonstrates about 3–10× differences across the GPU/SXM generations for this simple case (Table 1). UVA memory copy between GPUs will transparently fall back to a host-based copy if peer access is not available between devices.

4.1.3 Software: Our implementation uses CUDA and OpenMP [13] to take advantage of both GPUs and CPUs. We use CUDA 11.1 for the NVIDIA DGX platforms, whereas CUDA 10.2 on the IBM Power9 system as per the site-wide availability. We also used cuGRAPH v21.10 and built the Python dependencies using miniconda3.9.

4.1.4 Datasets and Analysis: We use graphs from the Suitesparse collection [28] and Graph Challenge repository [44]. Overall, the graphs are classified into two groups to perform distinct analyses:

- Multi-GPU evaluations: Mostly large graphs (with stable community structure) to assess baseline scalability (using a single batch), study performance profiles and the impact of batches on convergence, quality and overall performance (§4.2).
- Single-GPU evaluations: Small/medium sized graphs with relatively poor community structure or strong inter-cluster connections (challenging cases). We conduct comparative evaluations with related approaches, emphasizing on quality (§4.3).

To navigate the results better, we will begin each forthcoming section with a block that captures the range of research questions addressed within a specific section.

4.2 Multi-GPU evaluations

Scalability of single/multiple batch(es) on GPUs (compared to best CPU version), convergence characteristics on increasing batches, and performance profile studies for large graphs.

4.2.1 Baseline single-batch performance: We discuss the baseline performance (i.e., using a single batch, except one special case) of our implementation on multiple GPU platforms (refer to §4.1.1). The input consists of graphs obtained from the SuiteSparse Matrix Collection [28], as listed in Table 2. Table 2 also provides a performance summary of the best scalability achieved on multiple GPUs relative to a single GPU (on any platform) and over an optimized shared-memory version of the Louvain graph clustering, i.e., GRAPPOLO. Best CPU results were obtained at 128 (OpenMP) threads.

Since we do not use any other heuristic for handling parallel community updates (refer to §3.4), for certain graphs (with relatively poor structure and low modularity, like mycielskian20, which has no 3-cycle path a.k.a triangles) using a single batch can lead to an exit on the very first Louvain iteration. This is true for any parallel Louvain implementation that is not using coloring or vertex labeling schemes. Therefore, mycielskian20 is a special case, and we use 4 batches for it in the baseline evaluations.

Table 2 conveys the message that although our GPU implementations demonstrate scalability, for roughly 50% of the graphs considered, an optimal CPU implementation usually provides the best performance. Graph analytics in general is a challenging area for GPUs owing to irregular memory accesses and control flow [9]. In our implementation, to enable coalesced accesses to the graph data in a GPU, we perform a *sort* operation (refer to §3.3), which

Table 2: Graphs used in our GPU evaluations (ordered by #edges). Max speedup obtained relative to executions on a single GPU (across platforms) and 128 threads on an Intel Cascade Lake (CLX) CPU (using GRAPPOLO). Speedups < 1 demonstrate CPU advantage.

Graphs	V	E	Max speedup	
			over GPUs (A100/V100)	over CPU (Grappolo)
uk-2007-05	105.89M	6.6B	2.1×	0.27×
sk-2005	50.63M	3.86B	2.02×	0.27×
com-friendster	65.6M	3.61B	13.75×	3.41×
twitter7	41.65M	2.93B	1.5×	4.55×
it-2004	41.29M	2.27B	3.29×	0.83×
mycielskian20	786.43K	1.35B	3.92×	53.43×
webbase-2001	118.14M	1.98B	3.76×	0.97×
com-orkut	3.07M	234.37M	2.1×	1.31×

adds overhead (the performance implications would be discussed in upcoming §4.2.3). On shared-memory CPU platforms, simple graph representations such as CSR and efficient data structures available from C++ STL can often lead to optimal performance and scalability. However, since a group of threads are processed in a lockstep on a GPU, serialization of memory accesses can incur major overheads, especially for graph access patterns which are irregular. Fig. 3 shows the execution time performance across the GPU platforms. The graph aggregation time depends on the total #phases, and thus can be highly variable; we estimate it to be about 10–40% of the total elapsed time in most cases considered. Overall, we observed better scalability on NVIDIA V100 DGX-2 platform (about 2–14×). Whereas, the DGX-A100 platform exhibited about 30–40% better performance compared to previous generation DGX-2 V100 GPUs. The Power9 V100-SXM2 platform demonstrated the least scalability among the three.

4.2.2 Convergence and scalability: We now study the effect of batching on the convergence characteristics. Louvain is a multi-phase multi-iterative optimization method for graph clustering. Depending on the structure of the graph and desired tolerance, the number of phases until convergence can vary significantly. “Fundamentally, the mechanisms by which modularity emerges in complex networks are not well understood” [23]. To delineate the varying convergence characteristics, we separate the graphs that are well structured (modularities close to 1) from the ones that have modularities ≤ 0.7 . Fig. 8 depict cases which demonstrate minor difference in the convergence characteristics for batches greater than two. As a result, the overall execution times for larger batches are similar (Fig. 4).

Fig. 9 depict convergence characteristics of graphs with modularities close to 1. We observe that these graphs run for relatively more phases until convergence, and the choice of the number of batches can greatly impact the overall quality. We also make a note that for phases beyond the first phase for these graphs, the #iterations or loops are relatively low. As shown in Fig. 9, we are able to scale up to 16 batches; beyond that, the effects of per-batch sorting impacts the overall scalability.

As shown in Fig. 9 and Fig. 8, with more batches, the quality is also better across phases (in terms of modularity), which comes at the expense of performance in some cases. In Fig. 10, we address the topic of time-to-convergence for batches for the four largest

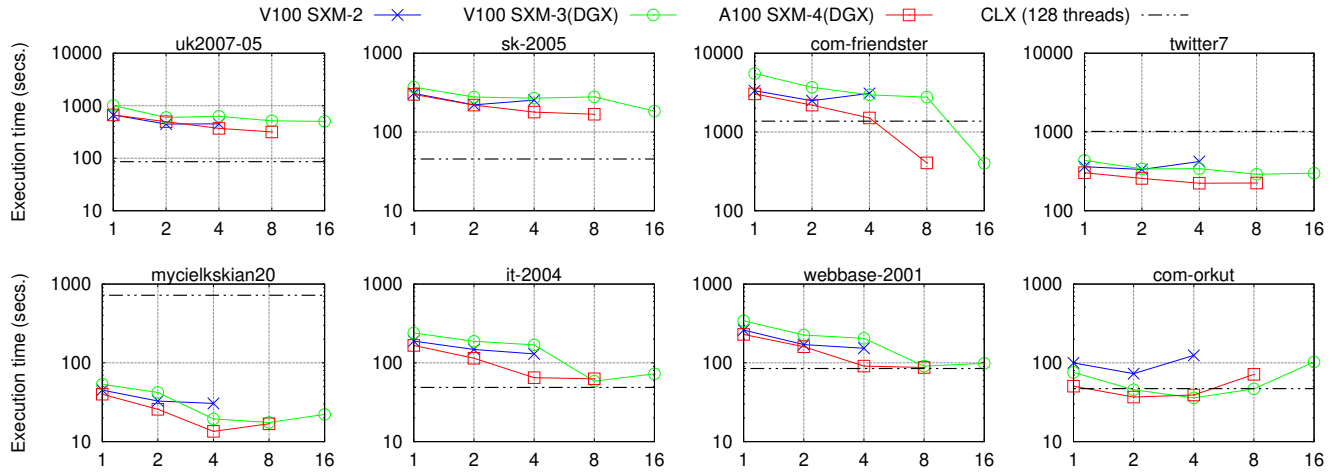


Figure 3: Baseline (single batch) execution times on 1–16 GPUs (X-axis: #GPUs); mycielskian20 is a special case, used 4 batches. Broken line indicates the best CPU performance on 128 threads of Intel Cascade Lake (CXL) platform using GRAPPOLO [33].

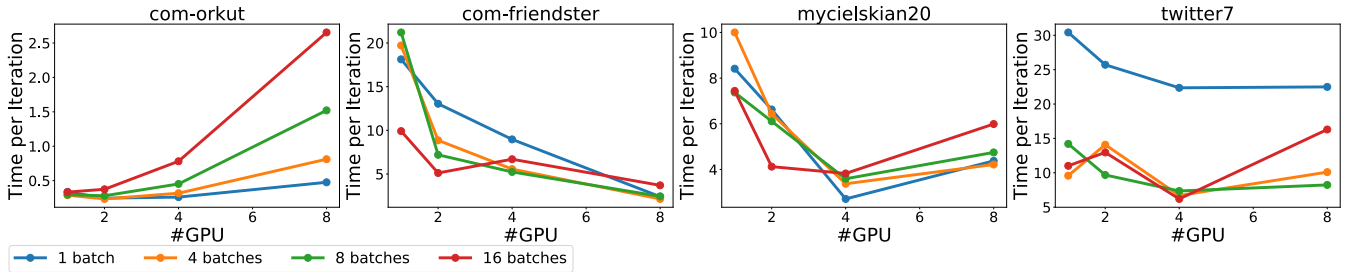


Figure 4: Execution time (secs.) for a Louvain iteration on DGX-A100 for graphs with relatively smaller modularities (≤ 0.7).

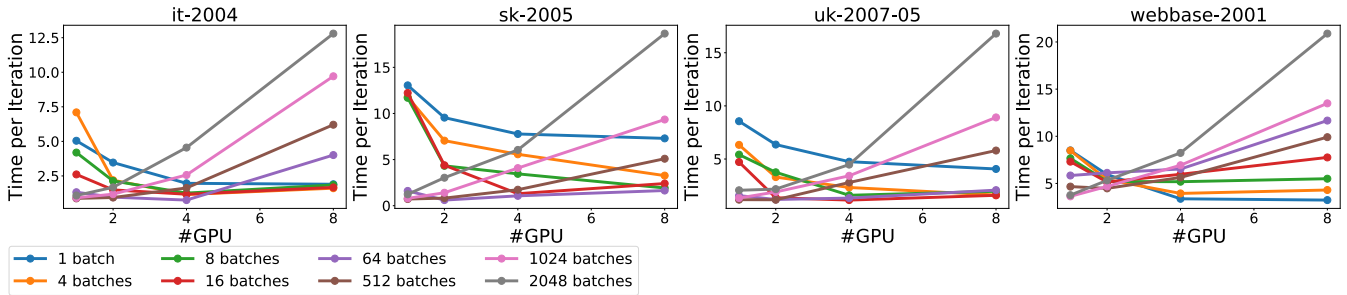
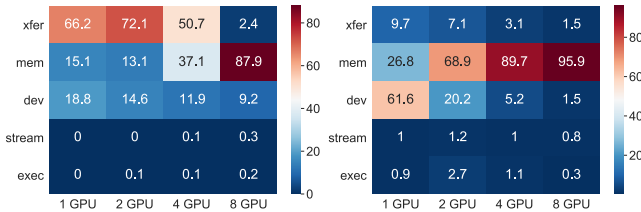


Figure 5: Execution time (secs.) for a Louvain iteration on DGX-A100 for graphs with relatively higher modularities.



(a) %-time CUDA, friendster (b) %-time CUDA, orkut
Figure 6: % of overall time in CUDA on 1–8 DGX-A100 GPUs for com-friendster and com-orkut inputs.

graphs. The first phase of Louvain is usually the most expensive (due to computations on the full uncompacted graph), and batches can reduce the iterations to convergence for the very first phase (leading to 20–45% improvements in time compared to a single batch), as shown in the sub-plots of Fig. 10. For graphs requiring more phases to converge (like uk2007-05 and sk-2005), we observe some fluctuations (up to 40% in some cases) in the later phases, whereas graphs converging within relatively less phases (like twitter7 and com-friendster) depict less variations. For graphs with poor community structure (like twitter7), it only takes relatively few iterations per phase to converge; although increasing batches can improve the overall quality by 40–60% (as shown in Fig. 8), it

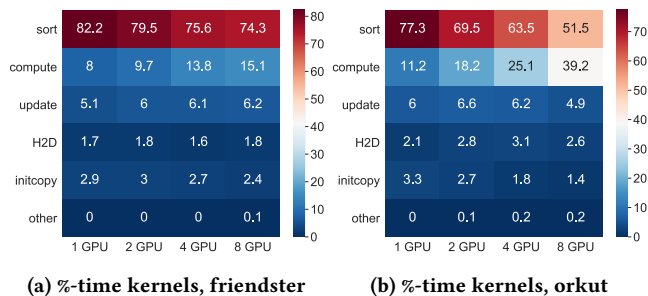


Figure 7: % of overall time for GPU kernels on 1–8 DGX-A100 GPUs for com-friendster and com-orkut inputs.

also increases the iterations per phase, thereby increasing the time with batches (refer to the sub-plot for twitter7 in Fig. 10).

4.2.3 Profiling analysis: To understand the bottlenecks in our GPU implementation, we study the multi-GPU profiles obtained using the NVIDIA™ Nsight™ Profiler on the DGX-A100 platform. We use two graphs for this analysis—com-friendster and com-orkut, which demonstrates significantly different scaling characteristics (as shown in Fig. 3). While com-friendster exhibits a “perfect” speedup of about 8× on 8 DGX-A100 GPUs, com-orkut is about 15× smaller, and demonstrates less than 2× speedup on 8 GPUs. Figures 6 and 7 depicts the overall % of time taken by CUDA functions and the device kernels. The profiles were obtained for runs using 8/16 batches.

As shown in Fig. 6a, for com-friendster about 90% of the time in CUDA is spent in memory allocation and transfer (as denoted by “xfer” and “mem”). And, about 10–20% of the time is spent in device synchronization (denoted by “dev”), arising due to the per-batch synchronization. For com-orkut, the overhead of memory allocation and data transfer increases with the #GPUs as shown in Fig. 6b (up to about 97% on 8 GPUs), suggesting little useful work. CUDA synchronization between device and host performs polling, i.e. a busy-wait loop, which involves the CPU and can be significant when there is load imbalance. As mentioned in §3, we perform a sort operation for every batch. Fig. 7 shows that about 50–80% of the time is spent in sorting, whereas the rest is spent equally between computing modularity and associated operations (denoted as “compute”), and updating the vertex-community information (depicted by “update”) in addition to host-device transfers, copying buffers, setup and initialization.

4.3 Single GPU evaluations

Compare quality of our batched GPU version against distance-1 coloring on CPU and a state-of-the-art vendor GPU implementation for challenging graphs on a single GPU.

In this section, we compare the quality of our approach with the existing parallel Louvain implementations. We use a set of graphs with community structures that can be challenging for Louvain implementations to detect the clusters. For instance, we consider a set of “challenge” graphs with known ground-truth partitions, that have a high-degree of block overlap and size variation, as a “hard” case for modularity maximization based graph clustering [27]. Community detection algorithms based on modularity maximization are susceptible to the *resolution limit problem*, where they cannot distinguish between two clearly defined clusters that are smaller than a

certain size with respect to the total size of the input (defined to be the square root of the total number of edges in the network) [19].

cuGRAPH is part of the NVIDIA RAPIDS toolkit [26] and is one of the fastest implementations of the Louvain method on a single GPU. GRAPPOLO [5, 33] is one of the optimized shared-memory implementations of Louvain method, that includes a number of heuristics such as coloring and vertex reordering. We use the coloring heuristic in GRAPPOLO to enhance the quality of clustering (which uses a parallel greedy distance-1 coloring discussed in [10]).

Input graphs used in the quality analysis are listed in Table 3, along with the results comparing our work with cuGRAPH and GRAPPOLO. We expect to provide a better quality compared to cuGRAPH, while remaining competitive to GRAPPOLO with coloring enabled (purportedly the best case). As shown in Table 3, although we incur a performance overhead due to processing in batches, we are able to provide similar (or better) quality as compared to GRAPPOLO. The #batches used for our implementation is included (inside parentheses) in Table 3 and range from 2–1024. The GPU results are obtained from a single DGX-A100 GPU, whereas we use the Intel CXL platform for GRAPPOLO with 128 OpenMP threads (§4.2.1). We compare the community assignments for the Graph Challenge “hard” cases to the ground-truth, and observe better F-scores (represents statistical similarity, closer to 1 is better) as compared to cuGRAPH in Table 4 (despite cuGRAPH exhibiting better modularities for the “hard” cases). Overall, Table 3 shows that our implementation using several batches on a single GPU performs better than distance-1 coloring enabled GRAPPOLO on more than 11/16 of the graphs considered, and on 12/16 graphs, the quality is better than cuGRAPH/GRAPPOLO.

5 Related work

Research on (shared/distributed memory) parallel Louvain method have been actively investigated [3, 15, 17, 22, 33, 42, 47, 49], however GPU implementations of Louvain method (and in general graph clustering) is still relatively sparse [11, 36, 40].

One of the first implementations of the Louvain method on multiple GPUs is discussed in [11]. The authors propose a hierarchical Louvain algorithm by splitting a graph into disconnected subgraphs and then invoking Louvain method on the subgraphs in parallel, before reconnecting the subgraphs and invoking Louvain again on a much coarser graph (similar to dividing the vertices and associated edges across processes in distributed-memory, but without the “ghost” processes). They obtain a speedup of 3–16× compared to the original serial Louvain implementation on Tesla C2070 GPUs. In [11], the best speedup obtained on 8 Tesla C2070 GPUs over a single GPU is 3.5×, whereas it is about 7.52× for us (on 8× A100). To be fair, the fastest GPU in our evaluation (A100) provides ~15× concurrency compared to Tesla C2070 (6912/448 cores) apart from other innovations. For com-orkut (our smallest input) we are ~8× better than [11] on a single GPU.

A single-GPU Louvain implementation is discussed in [36]. Unlike them, we do not perform the graph compaction on the device; compaction involves sorting the edges according to the new vertex indices, and potentially extra memory ($O(|E|)$). They also implement an adaptive threshold heuristic to improve performance. The authors show an average speedup of 40× relative to the serial

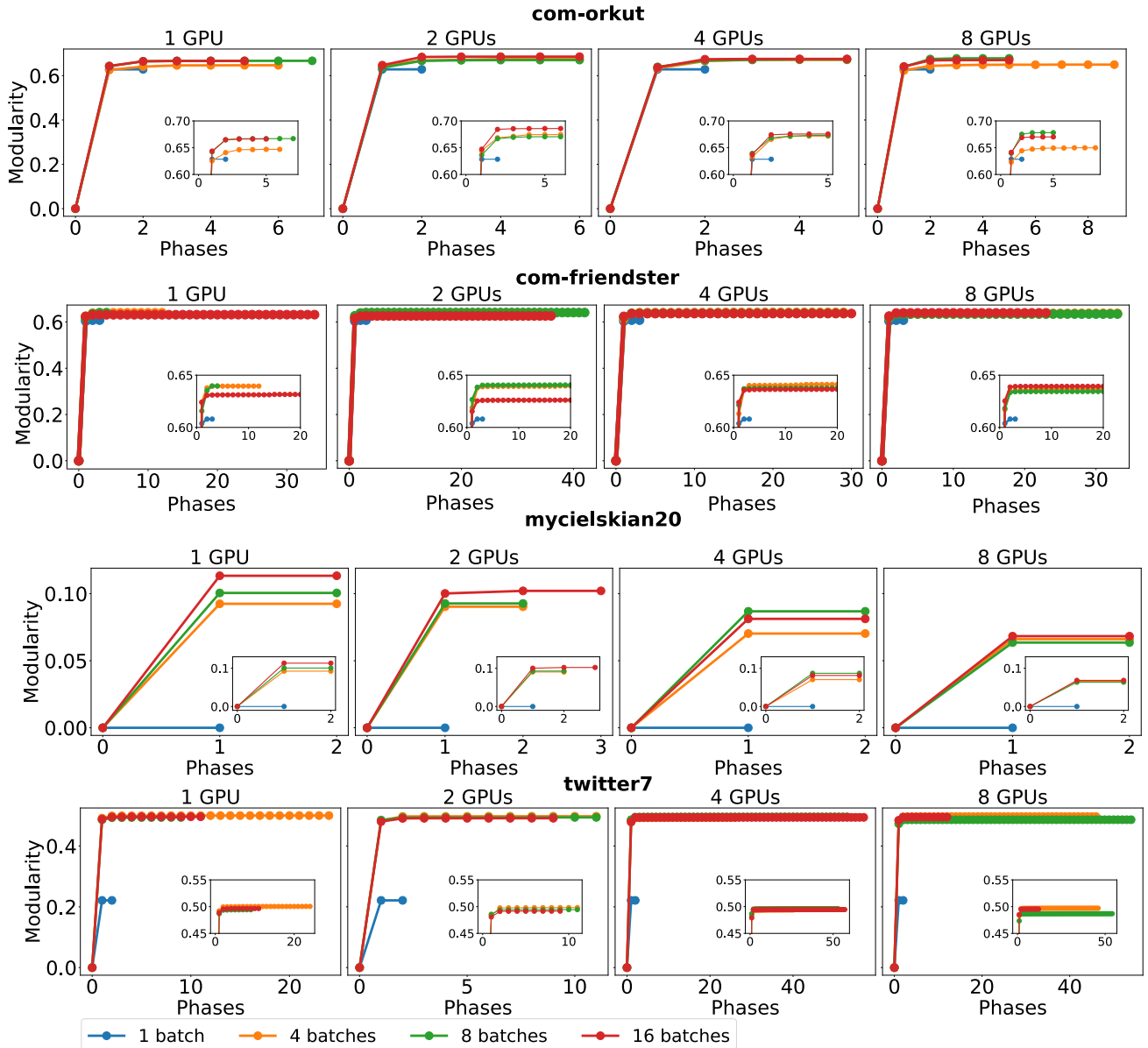


Figure 8: The modularity traces obtained from various numbers of batches on DGX-A100 are presented. The associated graphs has their final modularity lower than 0.7 and only a few first phases yield the modularity close to the final results.

algorithm and between 1.1–27 $\times$ compared to a shared-memory implementation (using 32 threads) on NVIDIA Tesla K40 GPU (a gaming GPU). As we have seen during our baseline evaluations (§4.2.1), performance depends on the input graph structure, and maintaining sustainable speedup over CPU platforms is difficult.

GPU enabled graph clustering is also part of some existing graph analytics toolkits, like GUNROCK [40] and NVIDIA RAPIDS cuGRAPH [39]. We used cuGRAPH in §4.3, and found it among one of the fastest single-GPU Louvain method implementations (especially for graphs with relatively low modularity and converges within a few phases). For instance, the time taken for cuGRAPH to run com-orkut on a DGX-2 A100 is about 8 seconds (at least 5 $\times$ faster

than our implementation and GRAPPOLO). The version of cuGRAPH Louvain that we used requires the difference between the current and previous modularities (ΔQ) to be at least 1.0E-03³, whereas for our implementation it is 1.0E-06. This can potentially lead to a faster convergence rate for cuGRAPH.

Multi-GPU cuGRAPH uses Dask [43] and exposes a newer version of Louvain (referred as MNMG) than the one for single-GPU (denoted as SG), that we have used previously. Other than com-orkut, cuGRAPH (SG and MNMG) ran into out-of-memory issues for the large graphs even on multiple GPUs. In Table 5, we compare

³<https://github.com/rapidsai/cugraph/blob/branch-21.10/cpp/src/community/louvain.cuh#L365>

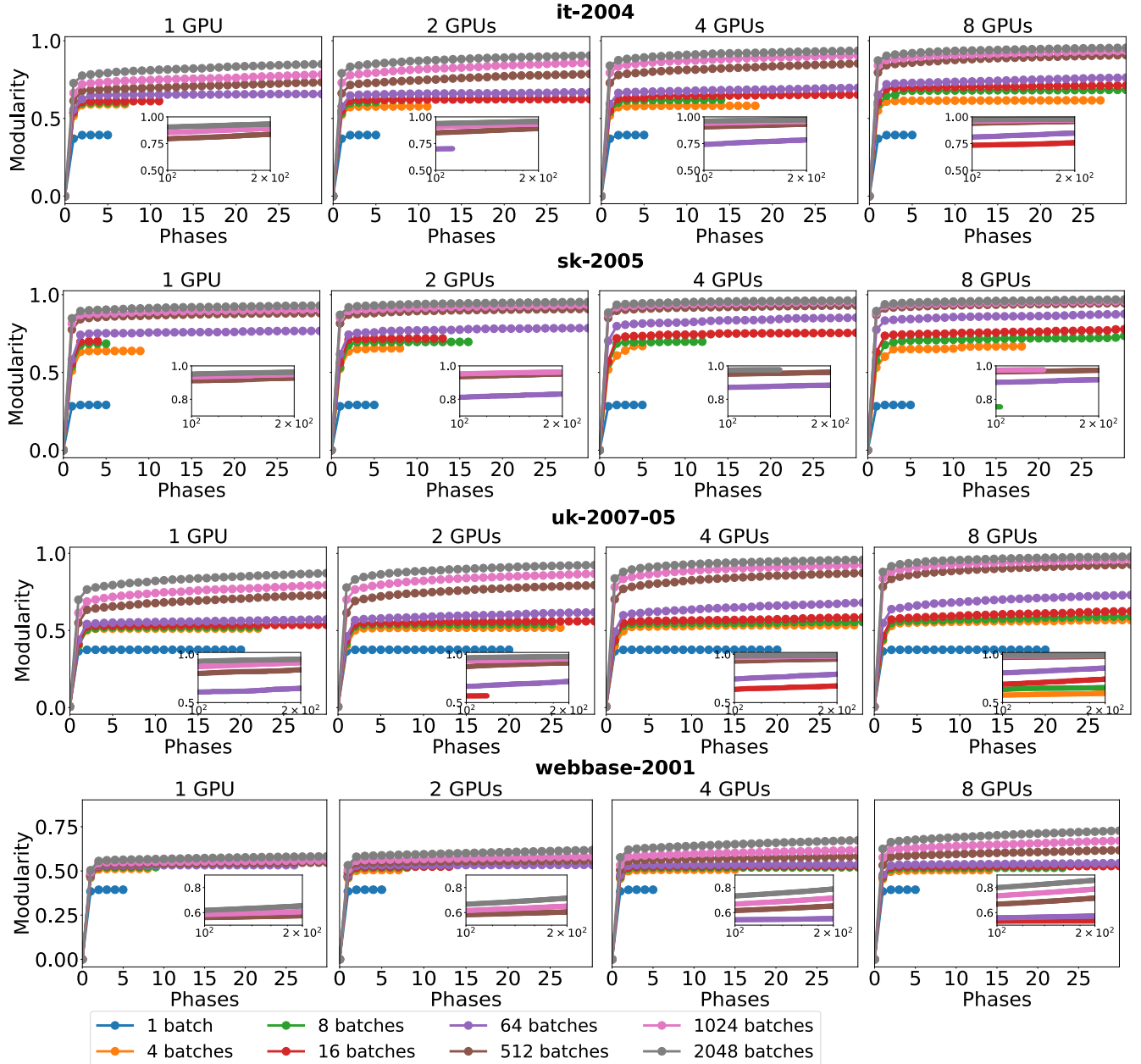


Figure 9: The graphs with higher final modularities require more phases to converge (experiments on DGX-A100). Batches contributed more toward improving the modularity as compared to the inputs on Fig. 8.

cuGRAPH(MNMG) Louvain performance with our implementation on the DGX-A100 platform, and observe about 1.5–30× better execution times.

6 Summary

Louvain method is a simple and fast unsupervised technique for identifying communities in graphs. Parallel implementations of the Louvain method require additional heuristics to mitigate performance/quality issues. We propose a new heuristic, i.e. *batching*, which has minimal overheads compared to distance- k coloring

and provides a flexible alternative to existing labeling heuristics. Our GPU memory-aware graph partitioning scheme is appropriate for supporting diverse graphs on memory-limited GPUs, and augments our batching methodology. Batching affects the quality (Fig. 9), however it also improves the performance (while maintaining scalability across multiple GPUs) by reducing the #iterations to convergence (Figures 4 and 9), demonstrating up to 45% performance improvement in the relatively expensive first phase (Fig. 10). For graphs that run for relatively large #phases till convergence, more #batches usually yield to better quality (sometimes at the

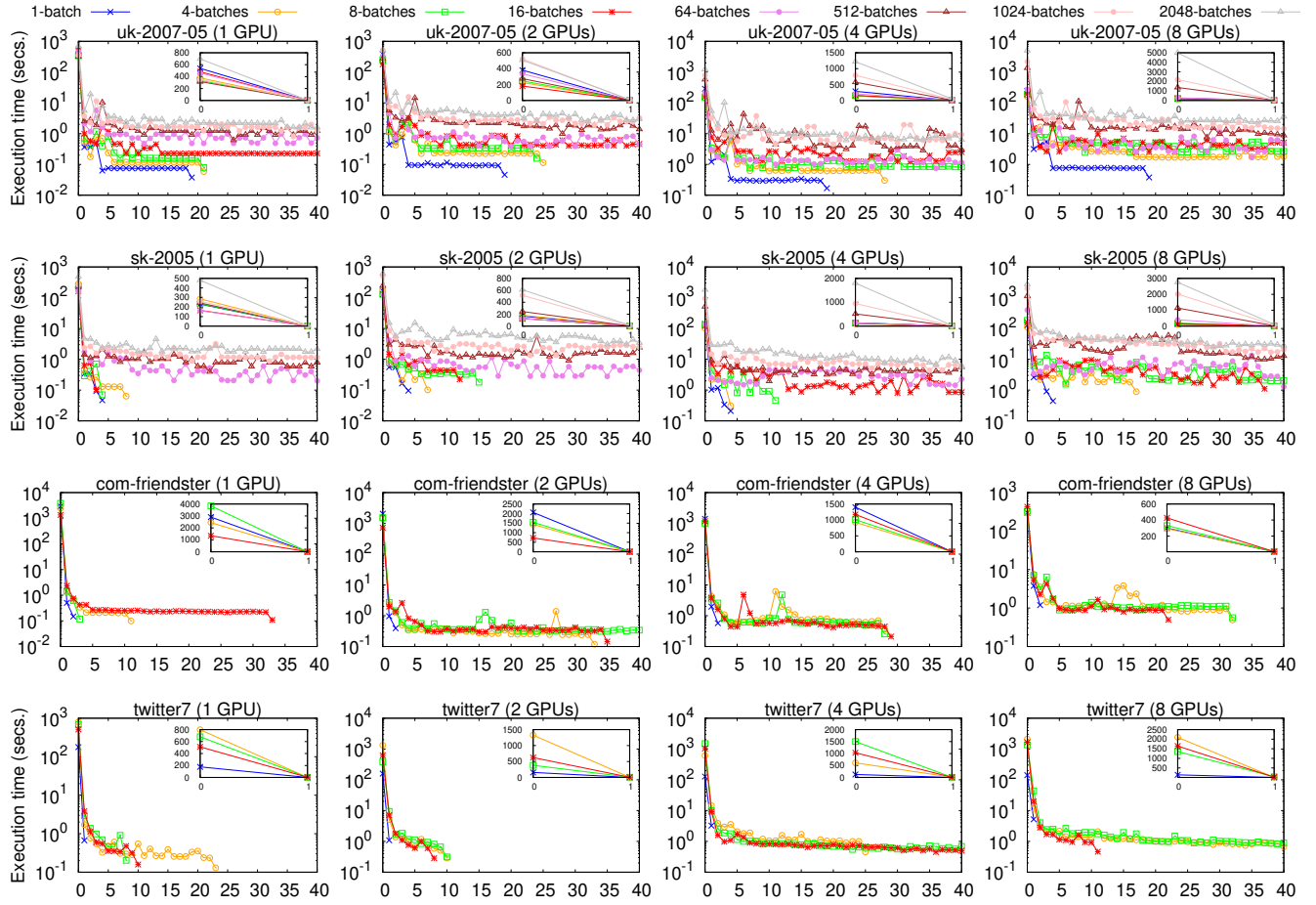


Figure 10: Execution time (secs.) per phase (X-axis: #Phases) for various number of batches on DGX-A100 platform for the four largest graphs (these graphs demonstrate different modularities and convergence characteristics, Fig. 8 and Fig. 9). The com-friendster and twitter7 graphs use 1–16 batches whereas uk-2007-05 and sk-2005 are evaluated using 1–2048 batches. Inset plot captures execution time differences between the first phase (most expensive) and the next for different batches.

expense of performance). Whereas, graphs that converge in less #phases does not exhibit better qualities with increasing #batches (Figures 8–9).

Our baseline multi-GPU Louvain method implementation demonstrated a maximum of about 14× speedup on 16× NVIDIA DGX-2 V100 GPUs and 7.5× speedup on 8× NVIDIA DGX-A100 GPUs, relative to a single GPU execution. We also observed maximum speedups of about 50× and 30× as compared to a state-of-the-art shared-memory and GPU Louvain implementations. Apart from performance, we provide competitive if not better quality as compared to greedy coloring heuristics. Our code is available in the following GitHub public repository: <https://github.com/sg0/nido>.

Contemporary AMD GPUs offer comparative programming abstractions as well (e.g., AMD HIP framework [31]), so it is possible to target AMD GPUs with reasonable efforts. We plan on further improving the performance of our batched Louvain implementations on single and multiple GPUs. With the next generation of GPU high-bandwidth memory (i.e., HBM3 is expected in 2022), we are looking at about 2–3× capacity with up to 3× aggregate bandwidth

as compared to the current generation. Increase in capacity will allow us to move more computations on the GPU (per partition), since we would be able to afford the memory.

Acknowledgment

This research was supported by the Data-Model Convergence (DMC), a multi-disciplinary Pacific Northwest National Laboratory (PNNL) initiative that applies co-design to develop heterogeneous computing capabilities to tackle challenging scientific problems. PNNL is operated by Battelle Memorial Institute under Contract DE-AC06-76RL01830.

References

- [1] JR Allwright, R Bordawekar, PD Coddington, K Dincer, and CL Martin. A comparison of parallel graph coloring algorithms. *SCCS-666*, pages 1–19, 1995.
- [2] Nathan Bell and Jared Hoberock. Thrust: A productivity-oriented library for cuda. In *GPU computing gems Jade edition*, pages 359–371. Elsevier, 2012.
- [3] Sanjukta Bhowmick and Sriram Srinivasan. A template for parallelizing the louvain method for modularity maximization. In *Dynamics On and Of Complex Networks, Volume 2*, pages 111–124. Springer, 2013.

Table 3: Comparing quality/performance for cuGRAPH, GRAPPOLO (with distance-1 coloring) and our implementation (including 2-batch results as a baseline, along with the batch that yielded the best quality). cuGRAPH/this work uses 1 NVIDIA A100 GPU, whereas GRAPPOLO uses 128 threads.

Graphs	V	E	cuGraph		This work		Grappolo	
			Q	Time(s)	Q(#batch)	Time(s)	Q	Time(s)
g500-s21	1.24M	31.73M	0.07	2.19	0.069 (2)/0.085 (4)	18.12 (2)/12.84 (4)	0.078	72.91
g500-s22	2.39M	64.09M	0.06	3.78	0.07 (2)/0.083 (4)	43.85 (2)/20.83 (4)	0.078	147.32
g500-s23	4.60M	129.25M	0.04	7.02	0.060 (2)/0.080 (8)	95.9 (2) / 88.26 (8)	0.076	426.90
MAWI-1	18.57M	19.02M	0.25	48.17	0.77 (2)/0.878 (32)	110.09 (2)/125.44 (32)	0.883	62.75
MAWI-2	35.99M	37.24M	0.35	175.09	0.74 (2)/0.855 (8)	558.67 (2)/372.38 (8)	0.880	210.12
MAWI-3	68.86M	71.70M	0.27	467.90	0.742 (2)/0.857 (32)	542.81 (2)/422.11 (32)	0.873	430.50
MAWI-4	128.56M	135.11M	0.21	1454.26	0.740 (2)/0.859 (32)	801.85 (2)/920.62 (32)	0.866	849.45
hihi_1K	1K	7.8K	0.50	0.03	0.391 (2)/0.473 (128)	0.036 (2)/0.50 (128)	0.512	0.09
hihi_5K	5K	50.8K	0.55	0.04	0.370 (2)/0.533 (8)	0.046 (2)/0.060 (8)	0.566	0.18
hihi_50K	50K	118.35K	0.64	0.09	0.631 (2)	0.27	0.643	1.37
hihi_200K	200K	474.7K	0.66	0.24	0.336 (2)/0.566 (8)	0.90 (2)/0.75 (8)	0.652	2.98
hihi_1M	1M	23.76M	0.26	1	0.333 (2)/0.458 (512)	3.65 (2)/5.52 (512)	0.505	25.11
hihi_5M	5M	118.76M	0.25	3.97	0.340 (2)/0.396 (1024)	26.66 (2)/115.13 (1024)	0.470	153.15
sx-stackoverflow	2.6M	36.23M	0.41	3.52	0.539 (2)	5.28	0.454	16.22
mycielskian17	98.3K	100.24M	0.0007	2.94	0.148 (2)	16.79	0.131	25.29
mycielskian18	196.6K	300.93M	0.0005	4.59	0.150 (2)	31.58	0.132	104.36

Table 4: F-score of community assignments compared to ground-truth.

	hihi_1K	hihi_5K	hihi_50K	hihi_200K
Grappolo	0.79	0.76	0.61	0.49
cuGraph	0.14	0.14	0.04	0.02
Ours	0.57	0.35	0.44	0.13

Table 5: Comparing cuGRAPH(MNMG) times (secs.) over Dask with our work on DGX-A100 (■ = out-of-memory).

Versions/#GPUs	1	2	4	8
com-orkut (E =234.37M)				
cuGRAPH(MNMG)	76.51	64.93	53.44	106.73
This work	50.51	36.79	39.1	71.4
mycielskian20 (E =1.35B)				
cuGRAPH(MNMG)	■	■	■	491.06
This work	40	25.73	13.49	16.86
webbase-2001 (E =1.98B)				
cuGRAPH(MNMG)	■	■	■	1339.83
This work	229.57	160.28	91	87.33

- Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefevre. Fast unfolding of communities in large networks. *Journal of statistical mechanics: theory and experiment*, 2008(10):P10008, 2008.
- Tom Bodenheimer, Mahantesh Halappanavar, Stuart Jefferys, Ryan Gibson, Siyao Liu, Peter J Mucha, Natalie Stanley, Joel S Parker, and Sara R Selitsky. Fastpg: Fast clustering of millions of single cells. *bioRxiv*, 2020.
- Joan Boyar, Leah Epstein, Lene M Favrholdt, Kim S Larsen, and Asaf Levin. Batch coloring of graphs. *Algorithmica*, 80(11):3293–3315, 2018.
- Ulrik Brandes, Daniel Dellling, Marco Gaertler, Robert Gorke, Martin Hoefer, Zoran Nikoloski, and Dorothea Wagner. On modularity clustering. *IEEE transactions on knowledge and data engineering*, 20(2):172–188, 2007.
- Thang Nguyen Bui and Curt Jones. Finding good approximate vertex and edge partitions is np-hard. *Information Processing Letters*, 42(3):153–159, 1992.
- Martin Burtscher, Rupesh Nasre, and Keshav Pingali. A quantitative study of irregular programs on gpus. In *2012 IEEE International Symposium on Workload Characterization (IISWC)*, pages 141–151. IEEE, 2012.
- Ümit V Çatalyürek, John Feo, Assefaw H Gebremedhin, Mahantesh Halappanavar, and Alex Pothén. Graph coloring algorithms for multi-core and massively multi-threaded architectures. *Parallel Computing*, 38(10-11):576–594, 2012.
- Chun Yew Cheong, Huynh Phung Huynh, David Lo, and Rick Siow Mong Goh. Hierarchical parallel algorithm for modularity-based community detection using gpus. In *European Conference on Parallel Processing*, pages 775–787. Springer, 2013.
- Michele Coscia, Fosca Giannotti, and Dino Pedreschi. A classification for community discovery methods in complex networks. *Statistical Analysis and Data Mining: The ASA Data Science Journal*, 4(5):512–546, 2011.
- Leonardo Dagum and Ramesh Menon. Openmp: an industry standard api for shared-memory programming. *IEEE computational science and engineering*, 5(1):

46–55, 1998.

- Jack J Dongarra, Hans W Meuer, Erich Strohmaier, et al. Top500 supercomputer sites. *Supercomputer*, 13:89–111, 1997.
- Mahmood Fazlali, Ehsan Moradi, and Hadi Tabatabaee Malazi. Adaptive parallel louvain community detection on a multicore platform. *Microprocessors and Microsystems*, 54:26–34, 2017.
- Denis Foley and John Danskin. Ultra-performance pascal gpu and nvidia interconnect. *IEEE Micro*, 37(2):7–17, 2017.
- Richard Forster. Louvain community detection with parallel heuristics on gpus. In *2016 IEEE 20th Jubilee International Conference on Intelligent Engineering Systems (INES)*, pages 227–232. IEEE, 2016.
- Santo Fortunato. Community detection in graphs. *Physics reports*, 486(3):75–174, 2010.
- Santo Fortunato and Marc Barthélemy. Resolution limit in community detection. *Proceedings of the National Academy of Sciences*, 104(1):36–41, 2007.
- Michael R Garey, David S Johnson, and Larry Stockmeyer. Some simplified np-complete problems. In *Proceedings of the sixth annual ACM symposium on Theory of computing*, pages 47–63, 1974.
- Assefaw Hadish Gebremedhin and Fredrik Manne. Scalable parallel graph coloring algorithms. *Concurrency: Practice and Experience*, 12(12):1131–1146, 2000.
- Sayan Ghosh, Mahantesh Halappanavar, Antonino Tumeo, Ananth Kalyanaraman, Hao Lu, Daniel Chavarria-Miranda, Arif Khan, and Assefaw Gebremedhin. Distributed Louvain Algorithm for Graph Community Detection. In *2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 885–895. IEEE, 2018.
- Roger Guimera, Marta Sales-Pardo, and Luis A Nunes Amaral. Modularity from fluctuations in random graphs and complex networks. *Physical Review E*, 70(2):025101, 2004.
- Gregory Gutin, Tommy Jensen, and Anders Yeo. Batched bin packing. *Discrete Optimization*, 2(1):71–82, 2005.
- M Harris and K Perelygin. Cooperative groups: Flexible cuda thread programming. <https://developer.nvidia.com/blog/cooperative-groups>, 2017.
- Todd Hricik, David Bader, and Oded Green. Using rapids ai to accelerate graph data science workflows. In *2020 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–4. IEEE, 2020.
- Edward Kao, Vijay Gadepally, Michael Hurley, Michael Jones, Jeremy Kepner, Sanjeev Mohindra, Paul Monticciolo, Albert Reuther, Siddharth Samsi, William Song, et al. Streaming graph challenge: Stochastic block partition. In *High Performance Extreme Computing Conference (HPEC)*, 2017 IEEE, pages 1–12. IEEE, 2017.
- Scott P Kolodziej, Mohsen Aznaveh, Matthew Bullock, Jarrett David, Timothy A Davis, Matthew Henderson, Yifan Hu, and Read Sandstrom. The suitesparse matrix collection website interface. *Journal of Open Source Software*, 4(35):1244, 2019.
- Douglas Kothe, Stephen Lee, and Irene Qualters. Exascale computing in the united states. *Computing in Science & Engineering*, 21(1):17–29, 2018.

- [30] Milind Kulkarni, Keshav Pingali, Bruce Walter, Ganesh Ramanarayanan, Kavita Bala, and L Paul Chew. Optimistic parallelism requires abstractions. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 211–222, 2007.
- [31] Evgeny Kuznetsov and Vladimir Stegailov. Porting cuda-based molecular dynamics algorithms to amd rocm platform using HIP framework: performance analysis. In *Russian Supercomputing Days*, pages 121–130. Springer, 2019.
- [32] Ang Li, Shuaiwen Leon Song, Jieyang Chen, Jiajia Li, Xu Liu, Nathan R Tallent, and Kevin J Barker. Evaluating modern gpu interconnect: Pcie, nvlink, nv-sli, nvswitch and gpudirect. *IEEE Transactions on Parallel and Distributed Systems*, 31(1):94–110, 2019.
- [33] Hao Lu, Mahantesh Halappanavar, and Ananth Kalyanaraman. Parallel heuristics for scalable community detection. *Parallel Computing*, 47:19–37, 2015.
- [34] Andrew Lumsdaine, Douglas Gregor, Bruce Hendrickson, and Jonathan Berry. Challenges in parallel graph processing. *Parallel Processing Letters*, 17(01):5–20, 2007.
- [35] Richard Murphy, Jonathan Berry, William McLendon, Bruce Hendrickson, Douglas Gregor, and Andrew Lumsdaine. DFS: A simple to write yet difficult to execute benchmark. In *2006 IEEE International Symposium on Workload Characterization*, pages 175–177. IEEE, 2006.
- [36] Md Naim, Fredrik Manne, Mahantesh Halappanavar, and Antonino Tumeo. Community detection on the gpu. In *2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 625–634. IEEE, 2017.
- [37] Mark EJ Newman. Communities, modules and large-scale structure in networks. *Nature physics*, 8(1):25, 2012.
- [38] Mark EJ Newman and Michelle Girvan. Finding and evaluating community structure in networks. *Physical review E*, 69(2):026113, 2004.
- [39] NVIDIA. Nvidia rapids cudagraph. <https://github.com/rapidsai/cugraph>, 2020.
- [40] Yuechao Pan. *Multi-GPU Graph Processing*. PhD thesis, University of California, Davis, 2019.
- [41] Mason A Porter, Jukka-Pekka Onnela, and Peter J Mucha. Communities in networks. *Notices of the AMS*, 56(9):1082–1097, 2009.
- [42] Xinyu Que, Fabio Checconi, Fabrizio Petrini, and John A Gunnels. Scalable community detection with the louvain algorithm. In *Parallel and Distributed Processing Symposium (IPDPS)*, 2015 IEEE International, pages 28–37. IEEE, 2015.
- [43] Matthew Rocklin. Dask: Parallel computation with blocked algorithms and task scheduling. In *Proceedings of the 14th python in science conference*, volume 130, page 136. Citeseer, 2015.
- [44] Siddharth Samsi, Vijay Gadepally, Michael Hurley, Michael Jones, Edward Kao, Sanjeev Mohindra, Paul Monticciolo, Albert Reuther, Steven Smith, William Song, et al. Graphchallenge. org: Raising the bar on graph analytic performance. In *2018 IEEE High Performance extreme Computing Conference (HPEC)*, pages 1–7. IEEE, 2018.
- [45] George M Slota and Siva Rajamanickam. Experimental design of work chunking for graph algorithms on high bandwidth memory architectures. In *2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 875–884. IEEE, 2018.
- [46] Apache Spark. Apache spark. Retrieved January, 17:2018, 2018.
- [47] Christian L Staudt and Henning Meyerhenke. Engineering parallel algorithms for community detection in massive networks. *IEEE Transactions on Parallel and Distributed Systems*, 27(1):171–184, 2015.
- [48] Dominic JA Welsh and Martin B Powell. An upper bound for the chromatic number of a graph and its application to timetabling problems. *The Computer Journal*, 10(1):85–86, 1967.
- [49] Charith Wickramaarachchi, Marc Frincu, Patrick Small, and Viktor K Prasanna. Fast parallel algorithm for unfolding of communities in large graphs. In *High Performance Extreme Computing Conference (HPEC)*, 2014 IEEE, pages 1–6. IEEE, 2014.
- [50] Michael Wolf, Daniel Dunlavy, Richard B Lehoucq, Jonathan W Berry, and Daniel Bourgeois. Tridata: High performance linear algebra-based data analytics. Technical report, Sandia National Lab.(SNL-NM), Albuquerque, NM (United States), 2018.